

附件（四）

深圳大学实验报告

课程名称： 操作系统

实验项目名称： 综合实验 1

学院： 计算机与软件学院

专业： 计算机科学与技术（创新班）

指导教师： 周明洋

报告人： 李文俊 学号： 2023150001 班级： 高性能

实验时间： 2026 年 04 月 23 日

实验报告提交时间： 2026 年 05 月 17 日

教务处制

一、实验目的与要求：

综合利用进程控制的相关知识，结合对 shell 功能的和进程间通信手段的认知，编写简易 shell 程序，加深操作系统的进程控制和 shell 接口的认识

二、方法、步骤：(说明程序相关的算法原理或知识内容，程序设计的思路和方法，可以用流程图表述，程序主要数据结构的设计、主要函数之间的调用关系等)

1. 核心知识点

1.1.进程创建与映像替换 API

本实验的核心在于在用户态利用系统调用模拟 Linux 命令解释器的生命周期闭环。命令执行机制高度依赖进程控制 API: 父进程通过调用 fork 产生拥有独立虚拟地址空间的子进程, 子进程继而调用 execvp 族函数。选用 execvp 而非其他族系, 是因为其支持数组传参且能自动解析内核环境变量中的路径搜索列表, 完成程序代码段和数据段的映像无缝替换。为了防范僵尸进程的产生, 父进程必须调用 waitpid 陷入阻塞, 安全回收子进程的进程控制块残留状态。

1.2.文件描述符表与重定向底层机制

进程间通信与重定向机制重度依赖文件描述符的狸猫换太子技术。内核为每个进程维护了一张文件描述符表, 通过调用 pipe 在内核中开辟无名管道环形缓冲区并获取读写端描述符, 随后利用 dup2 系统调用将子进程的标准输入或标准输出强制覆盖为管道端口或由 open 以指定权限 (如 O_CREAT|O_TRUNC) 创建的磁盘文件描述符, 从而在目标程序无感知的情况下透明地改变了字节流的输送流向。

2. 实验设计思路

(1) **宏观总体调度流设计**: 整个定制 Shell 的生命周期被设计为一个标准的“REPL (读取-求值-输出)”模型。

每次循环伊始, 系统首先捕获当前路径构建高级提示符并阻塞等待用户输入。获取输入后, 程序按优先级依次探测管道符 (|)、重定向符 (>) 以及常规命令, 进而实施分流调度。主进程在分发完外部任务后将陷入挂起等待, 直至所有子进程完成工作, 从而实现生命周期的安全闭环。

(2) **死循环守护与输入清洗机制**: 程序的宏观架构被设计为一个死循环守护进程。输入读取采用 fgets 以防范缓冲区溢出, 并通过 strcspn 主动过滤并销毁末尾换行符, 防止脏字符污染后续的系统调用参数数组。

(3) **内外命令的状态隔离底线**：架构最核心的约束条件在于内外部命令的严格隔离。凡是改变当前执行环境状态的动作（如 exit、help）必须拦截并在父进程上下文中直接执行，以此确保环境状态的变更真实生效，而不被抛弃在死去的子进程遗骸中。

(4) **容错拦截与死锁防范体系**：防错机制是系统级编程的基石。当用户键入无效命令导致 execvp 替换失败返回时，子进程必须被强制调用 exit 自杀，切断其继续向下执行父进程循环逻辑的可能；在管道功能设计中，父进程在完成两个分身的派生后，必须毫无保留地立刻关闭自身持有的管道读写两端，防范读端子进程因等不到 EOF 信号而陷入永久死锁阻塞。

3. 流程图表示

Shell 的核心调度生命周期与条件分发逻辑如下流程图所示：

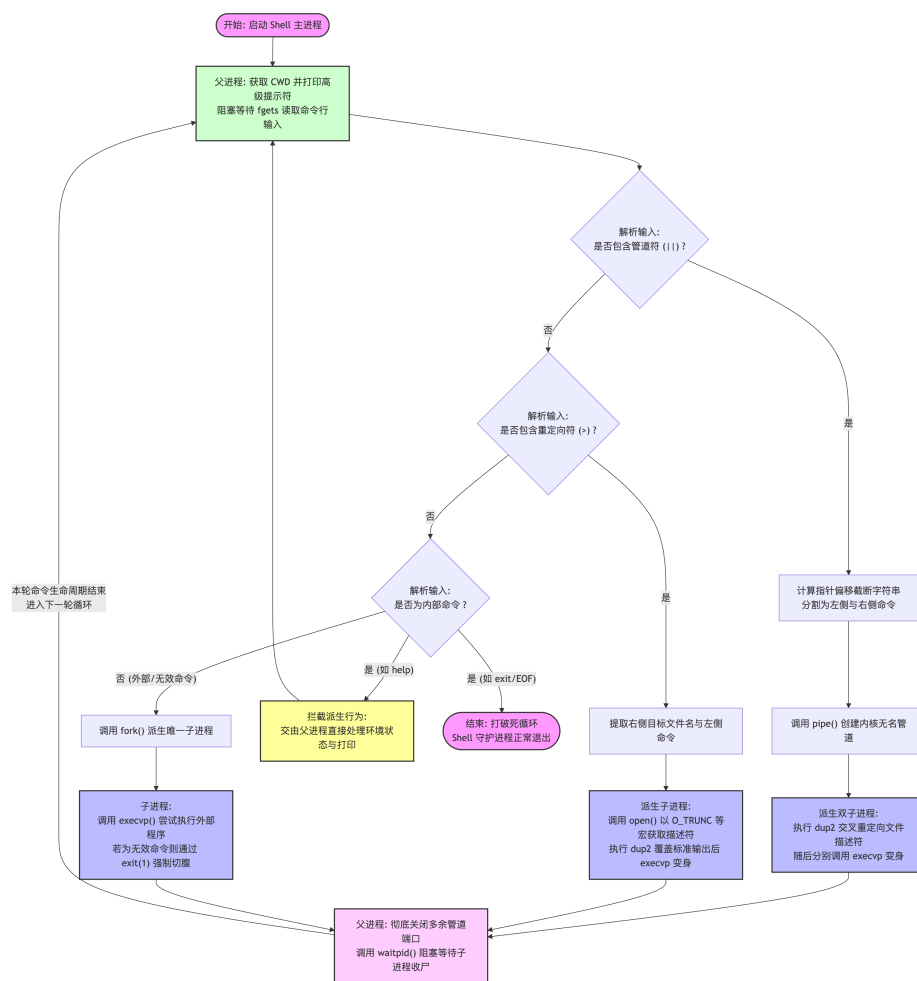


图 1 Shell 实现流程图

三. 实验过程及内容：(对程序代码进行说明和分析，越详细越好，代码排版要整齐，可读性要高)

1. 实验编译与运行环境部署

1.1 编译指令

实验基于标准 Linux 内核环境进行编译部署。为了防范多命令并行执行产生的旧版本覆盖问题，采用严格的短

路与逻辑控制编译与执行时序。终端部署指令为：

- `gcc -Wall -o myshell myshell.c && ./myshell`

利用逻辑控制符确保只有在 GCC 编译无任何警告与致命错误并返回零值后，内核才会拉起最新构建的二进制解释器进程

2. Shell 基础框架与命令调度机制深度解析

Shell 程序的基石是一个承载“读取-解析-分发-回收”生命周期的无限守护循环。对于常规命令（包含内部命令、外部命令及无效命令），其实施细节与底层动作剖析如下：

2.1 高级提示符构建与安全输入机制

程序主体运行在一个 while(1)死循环中。每轮循环伊始，**系统调用 `getcwd` 捕获当前绝对工作路径**，拼装成 `[MyShell:/path]$` 的高级提示符。

由于 `printf` 打印提示符时未附加换行符，必须显式调用 `fflush(stdout)` 强制清空标准输出缓冲区，以防提示符被内核滞留在内存中引发显示延迟。随后，调用 `fgets` 从标准输入安全读取定长字符，并通过 `strcspn` 精准定位并抹除用户键入的尾部换行符 `\n`，防止脏字符破坏后续 `execvp` 的参数寻址：

```
if (getcwd(cwd, sizeof(cwd)) != NULL) {
    printf("[MyShell:%s]$ ", cwd);
}
fflush(stdout); // 强制刷出内存缓冲区的提示符

// 捕获Ctrl+D产生的EOF信号，实现Shell进程的优雅退出
if (fgets(cmd_str, MAX_CMD_LEN, stdin) == NULL) {
    printf("\nExiting MyShell...\n");
    break;
}
cmd_str[strcspn(cmd_str, "\n")] = '\0'; // 物理抹除回车换行符
```

图 2 高级提示符构建与安全输入机制代码

2.2 参数切分与起始标签的统一包裹

获取纯净的命令字符串后，利用 `strtok` 函数以空格为定界符，将连续的字符串切分为离散的命令与参数，依次存入 `args` 指针数组，并严格在数组末尾封口 `NULL` 以符合系统调用规范。

为完美契合实验对于动作标识包裹的要求，在进入真正的命令路由分派前，由未受任何污染的父进程统一打印起步标签：`printf("%s starting...\n", args[0]);`

```

// 定义一个指针数组，用来存放切割后的命令和参数。
// 比如输入 "ls -l", 切分后 args[0]="ls", args[1]="-l", args[2]=NULL
char *args[MAX_ARGS];

// 1. 字符串切割（分词）
int i = 0;
// strtok 是 C 语言自带的切割函数，这里以空格 " " 为刀，切下第一块肉
char *token = strtok(cmd_str, " ");
while (token != NULL)
{
    args[i] = token; // 把切下来的字符串存入 args 数组
    i++;
    // 继续对剩下的字符串切片。strtok 规定后续切片第一个参数必须传 NULL
    token = strtok(NULL, " ");
}
args[i] = NULL; // Linux 规定，给 exec 执行的参数数组，最后一个元素必须是 NULL，代表参数结束

```

图 3 参数切分代码

2.3 三态命令的隔离调度与防御性执行

获取参数表后，程序通过严格的条件路由实现命令的三态隔离调度：

内部命令（如 help、exit）：此类命令的核心在于必须改变或读取 Shell 父进程自身的环境状态。因此直接拦截并在父进程上下文中处理。执行完毕后即刻打印 ending 标签，随后通过 continue 或 break 流转系统状态。

外部命令与无效命令：凡非内部命令，一律触发 fork 派生唯一的子进程进行尝试。子进程利用 execvp 在系统的环境变量 PATH 中检索目标程序并实施内存映像替换：

- **常规外部命令（如 ls、HelloWorld）：**若 execvp 成功，子进程原有的 Shell 代码段被彻底覆盖，转而执行目标程序，执行完毕后正常消亡
- **无效命令（如乱码错误输入）：**若用户键入不存在的命令，execvp 检索失败并返回 -1。此时触发极为关键的系统安全防御底线：子进程调用 perror 抛出系统报错后，必须立即调用 exit(1) 退出，绝对阻断其顺延执行父进程死循环的隐患

最后，无论子进程是正常结束还是因无效命令异常退出，挂起在 waitpid 处的父进程都能安全为其回收，并执行统一的收尾动作

```

pid_t pid = fork();
if (pid < 0) {
    perror("Fork failed");
} else if (pid == 0) {
    // 子进程尝试变身执行外部程序
    if (execvp(args[0], args) == -1) {
        perror("无效命令");
        exit(1); // 防御底线：变身失败的重伤子进程必须立刻终结
    }
} else {
    // 父进程挂起阻塞，尽到等待与收尸的职责
    waitpid(pid, NULL, 0);
    // 无论是外部任务顺利完成，还是无效命令报错收场，父进程统一在此打印结束标识
    printf("%s ending.\n", args[0]);
}

```

图 4 子进程执行指令，父进程等待代码

3. 自定义管道(|)的核心代码深度解析

针对“dir || more”等包含管道符的复合指令，代码实现了从字符串物理截断、参数解析到内核通道绑定的完整流程。具体操作逻辑剖析如下：

3.1 基于指针运算的字符串切割与分词

在检测到双竖线管道符后，程序直接在原始内存块上进行物理截断：

```
char *pipe_str = strstr(cmd_str, "||");
// 核心截断动作：将第一个'|'覆写为字符串结束符'\0'，左侧命令至此安全截断
pipe_str[0] = '\0';
char *cmd1_str = cmd_str;           // 左半部分命令起点
// 指针向后偏移2字节(跳过原本的两个'|')，精准定位右半部分命令起点
char *cmd2_str = pipe_str + 2;
```

图 5 物理截断代码

随后，利用 `strtok` 函数分别对 `cmd1_str` 与 `cmd2_str` 以空格为定界符进行循环分词，分词代码与普通命令相同，将拆解后的命令片段分别填入 `args1` 与 `args2` 指针数组，并在数组末尾追加 `NULL`，以严格满足 `execvp` 的参数表规范。

3.2 内核通道开辟与写入端（子进程 1）重定向

父进程首先调用 `pipe(fds)` 在内核空间开辟无名管道，获得读端 `fds[0]` 与写端 `fds[1]`，随后派生出负责处理左侧命令的子进程 1，利用 `dup2` 将子进程的 1 号端口(标准屏幕输出)强制覆盖并指向管道的写入端 `fds[1]`。

```
pid_t pid1 = fork();
if (pid1 == 0) {
    // 狸猫换太子：将本进程的1号端口(标准屏幕输出)强制覆盖并指向管道的写入端fds[1]
    dup2(fds[1], STDOUT_FILENO);

    // 资源防泄漏：换绑完成后，从父进程继承来的原始管道句柄对本进程已无用，必须立即关闭
    close(fds[0]);
    close(fds[1]);

    // 变身执行：此时args1程序的任何标准输出都会被静默压入内核管道
    if (execvp(args1[0], args1) == -1) {
        perror("无效命令");
        exit(1); // 若外部命令不存在，触发防御机制强制退出
    }
}
```

图 6 写入端（子进程 1）重定向代码

3.3 读取端（子进程 2）重定向与父进程收尾

紧接着，父进程再次派生负责处理右侧命令的子进程 2，完成数据管线的闭环；注意父进程也得同步关闭管道的读端和写端，具体原因在管道的预备知识学习中已经详细分析过。

```

pid_t pid2 = fork();
if (pid2 == 0) {
    // 反向重定向: 将本进程的0号端口(标准键盘输入)强制指向管道的读取端fds[0]
    dup2(fds[0], STDIN_FILENO);

    // 同样执行严格的句柄清理
    close(fds[0]);
    close(fds[1]);

    // 变身执行: 此时args2程序(如more)会由于标准输入被替换, 转而从管道中阻塞吸取数据
    if (execvp(args2[0], args2) == -1) {
        perror("无效命令");
        exit(1);
    }
}

/* 父进程的极其关键的调度职责 */
// 父进程自身不参与数据搬运, 若不关闭写端fds[1], 子进程2将永远无法收到EOF结束符, 进而引发永久死锁!
close(fds[0]);
close(fds[1]);

// 挂起阻塞, 严格按序回收两个子进程的PCB资源
waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);

```

图 7 子进程 2 重定向与父进程收尾代码

4. 自定义管道(|)的核心代码深度解析

对于“ls > a.txt”类指令，其核心在于将程序的输出流引导至磁盘物理文件。代码解析流程如下：

4.1 命令与目标文件名的剥离

剥离逻辑和管道的剥离一致，在检测'>'后，程序直接在原始内存块上进行物理截断：

```

char *redir_str = strstr(cmd_str, ">");
redir_str[0] = '\0'; // 将">"替换为结束符, 左侧变为纯净命令
char *cmd_part = cmd_str;
char *file_part = redir_str + 1; // 右侧提取出文件名区域
char *filename = strtok(file_part, " "); // 剥离多余空格, 获取纯净文件名

```

图 8 物理截断代码

4.2 文件句柄获取与单向流重定向

父进程派生单一子进程后，子进程通过 open 获得文件描述符后，经过重定向接管文件系统的 I/O 控制权：

```

pid_t pid = fork();
if (pid == 0) {
    // 核心安全策略：以只写模式打开。若文件不存在则自动创建(O_CREAT)，若存在
    // 则彻底清空历史数据(O_TRUNC)
    // 0644确保了创建出的文件具备标准的属主读写、属组及其他只读权限
    int fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) {
        perror("文件打开失败");
        exit(1);
    }

    // 将本进程的1号端口(标准输出)强制覆盖为刚刚获取的磁盘文件句柄fd
    dup2(fd, STDOUT_FILENO);
    close(fd); // 替换完成后关闭多余描述符防泄漏

    // 执行变身，原命令的屏幕输出被安全转储至磁盘文件中
    if (execvp(args[0], args) == -1) {
        perror("无效命令");
        exit(1);
    }
}
}

```

图 9 文件重定向代码

5. 管道与重定向的底层机制对比总结

通过上述代码的纵向比对，可清晰发掘两者在 Linux 内核态的实现异同：

- (1) **数据归宿与介质差异**：重定向(>)的本质是进程与磁盘文件系统的数据交互，输出流的最终归宿是持久化的物理存储介质；而管道(|)的本质是进程与进程间的 IPC 内存通信，数据完全在操作系统内核维护的内存环形缓冲区中流转，不涉及任何磁盘 I/O 开销。
- (2) **进程拓扑模型差异**：重定向模型中，系统仅需维持“父进程-**单子进程**”的二级拓扑，单子进程依靠自身打开文件获取描述符进行自我流替换；管道模型则必须构建“父进程-**双子进程**”的三级拓扑，父进程作为桥梁负责在内核开辟通道，并由两个兄弟子进程分别认领读写两端完成对接。
- (3) **资源死锁敏感度差异**：重定向的文件描述符为进程私有拥有，随进程消亡而自动闭合，几乎无死锁风险；管道描述符则被父子三进程共同持有（引用计数机制），一旦父进程漏写 close 导致写端引用计数始终大于 0，读端进程将陷入永久阻塞，其死锁敏感度远高于普通重定向。

四、实验结论：（提供运行结果，对结果进行探讨、分析、评价，并提出结论性意见和改进想法）

1. 运行结果

1.1 三种常规命令验证

程序运行后成功打印包含当前绝对路径的高级提示符，分别输入内部命令 `help`，外部命令 `ls` 和 无效命令 `asdb`，成功输出对应的正确结果：

```
wenjun@2023150001:~/intergrated_exp1/real_exp$ ./myshell
[MyShell:/home/wenjun/intergrated_exp1/real_exp]$ help
help starting...
--- 欢迎使用 MyShell ---
支持内部命令：help, exit
支持外部命令：ls, cp, pwd 以及当前目录下的可执行文件
help ending.
[MyShell:/home/wenjun/intergrated_exp1/real_exp]$ ls
ls starting...
hello hello.c myshell myshell.c test.c test.txt
ls ending.
[MyShell:/home/wenjun/intergrated_exp1/real_exp]$ asdb
asdb starting...
无效命令：No such file or directory
asdb ending.
[MyShell:/home/wenjun/intergrated_exp1/real_exp]$
```

图 10 三种常规命令运行结果

1.2 管道结果验证

键入外部查询指令联合管道分页指令，屏幕准确输出转移数据状态标签：

```
[MyShell:/home/wenjun/intergrated_exp1/real_exp]$ ls || grep myshell
ls || grep myshell transferring data...
ls starting...
grep starting...
myshell
myshell.c
ls ending.
grep ending.
ls || grep myshell finish data.
[MyShell:/home/wenjun/intergrated_exp1/real_exp]$
```

图 11 管道命令运行结果

1.3 输出重定向验证

键入外部查询指令联合重定向指令 `>`，屏幕准确输出转移数据状态标签，最后键入内部命令 `exit` 成功退出程序

```
[MyShell:/home/wenjun/intergrated_exp1/real_exp]$ ls > test.txt
ls starting...
ls ending.
[MyShell:/home/wenjun/intergrated_exp1/real_exp]$ cat test.txt
cat starting...
hello
hello.c
myshell
myshell.c
test.c
test.txt
cat ending.
[MyShell:/home/wenjun/intergrated_exp1/real_exp]$ exit
exit starting...
Bye!
exit ending.
wenjun@2023150001:~/intergrated_exp1/real_exp$
```

图 12 输出重定向运行结果

上述真实执行序列证实了定制解释器能够完美拦截并切割复合命令，且通过在父进程中提前打印起止标识的策略，成功保障了屏幕输出界面的整洁一致性，验证了父子多进程在复杂场景下调度时序的准确无误。

2. 预备 1-进程控制

2.1 进程基本概念与组织关系

2.1.1 观察进程状态与树状关系

在 Linux 终端中运行带有 `getchar()` 阻塞逻辑的 HelloWorld 程序，当执行命令并回车后，当前的 bash Shell 进程发现该命令非内部指令，便通过 `fork()` 创建子进程并用 `exec()` 执行可执行文件。此时，Shell 进程自动调用 `wait()` 进入休眠阻塞状态以交出终端控制权。

通过另一个独立终端执行进程查询命令，可以清晰观测到目标程序处于 **S+** 状态（S 代表 **Interruptible Sleep** 即可中断休眠，+ 代表其属于前台进程组）。

当启动三个 HelloWorld-getchar 程序后运行：`ps -aux | grep HelloWorld`

```
● liwenjun@liwenjun-VMware-Virtual-Platform:~/OS/Integrate_1$ ps -aux | grep HelloWorld
liwenjun  6378  0.0  0.0  2684  1180 pts/2    S+   13:22   0:00  ./HelloWorld-getchar
liwenjun  6410  0.0  0.0  2684  1180 pts/1    S+   13:23   0:00  ./HelloWorld-getchar
liwenjun  6454  0.0  0.0  2684  1180 pts/4    S+   13:23   0:00  ./HelloWorld-getchar
liwenjun  6502  0.0  0.0  6864  2212 pts/3    S+   13:23   0:00  grep  --color=auto HelloWorld
○ liwenjun@liwenjun-VMware-Virtual-Platform:~/OS/Integrate_1$
```

图 13 ps -aux 成功输出进程组织关系的画面

符号 `--` 表示直接的单线父子关系，比如 `bash--pstree`，最左侧的 `bash` 是父进程，它创建了右侧的 `pstree` 子进程，编写一个简单的 `fork` 创建子进程程序，可以看到 `bash--fork-demo--fork-demo` 完美展示了三代进程关系；`3*[bash--HelloWorld-getc]` 表示这里同时存在 3 个并列的 `bash` 父进程，并且这 3 个 `bash` 各自都拉起了一个 `HelloWorld-getc` 子进程

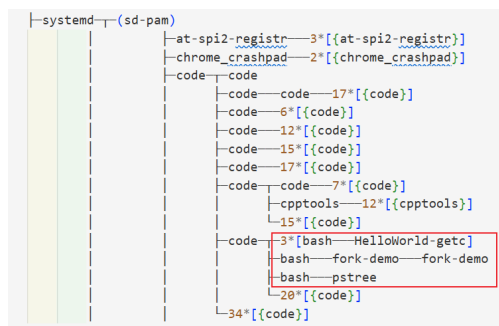


图 14 pstree 成功输出进程组织关系的画面

2.1.2 观察进程组与会话关系

在 Linux 操作系统中，“会话、进程组、线程组”构成了层级严密的任务调度与资源管理模型：

(1) **会话 (Session)**：是操作系统为统一管理用户登录活动而引入的顶层抽象。每当用户开启终端或建立远程连接时，系统便创建一个独立会话。会话的领头进程（Session Leader，通常为 Shell 进程本身）的 PID 会被直接指定为整个会话的标识符（SID），会话具备生命周期绑定特性，当控制终端断开或会话领头进程终止时，内核将触发级联资源回收逻辑，向内部派发 `SIGHUP` 挂断信号。

(2) **进程组 (Process Group)**：是构成会话的次级执行实体，主要服务于作业控制。一个独立命令或由管道连接的复合任务序列（如 `top | more`）会被系统自动打包为同一进程组，共享唯一的进程组 ID（PGID）。

进程组严格区分前台与后台两类：前台进程组独占终端控制权，能够批量接收内核因用户键入控制键（如 Ctrl-C、Ctrl-Z）而转换广播的 SIGINT 或 SIGTSTP 信号；后台进程组（通过命令末尾追加 “&” 触发）则被剥夺终端读写权限，若其违规发起终端 I/O 操作，内核将默认派发 SIGTTIN 或 SIGTTOU 信号将其强制挂起，以维护终端 I/O 的稳定性。

(3) 线程组 (Thread Group)：是进程组内部的基础并发调度单元。在 Linux 底层的设计哲学中，一个标准进程实体本质上就是一个单一线程的线程组。当进程通过调用 pthread_create 派生出多个轻量级进程时，它们共同组成一个线程组，并透明地共享所属主进程的核心系统资源（如相同的虚拟地址空间与文件描述符表），从而在极小的系统开销下支撑高密度的并发任务流

为验证 Linux 系统中会话与进程组的嵌套管理机制，在终端执行 “top | more &” 将其挂入后台运行，随后执行 “ps j | more”

```
liwenjun@liwenjun-Virtual-Platform:~/05/Integrate_1$ top | more &
[1] 11064
liwenjun@liwenjun-Virtual-Platform:~/05/Integrate_1$ ps j | more

[1]+ 已停止                  top | more
PPID    PID    PGID    SID    TTY      TPGID  STAT   UID    TIME  COMMAND
1546    1811   1811    1811   tty4      1811  Ssl+   1000    0:00  /usr/libexec/gdm-wayla
nd-session env GNOME_SHELL SESSION_MODE=ubuntu /usr/bin/gnome-session --session=ubuntu
1811    1819   1811    1811   tty4      1811  Sl+    1000    0:00  /usr/libexec/gnome-ses
sion-binary --session=ubuntu
2921    2929   2929    2929   pts/0      2929  Ss+    1000    0:00  bash
3549    3566   3566    3566   pts/1      6410  Ss     1000    0:00  /usr/bin/bash --init-f
ile /snap/code/194/usr/share/code/resources/app/out/vs/workbench/contrib/terminal/common/
scripts/shellIntegration-bash.sh
3549    3789   3789    3789   pts/2      6378  Ss     1000    0:00  /usr/bin/bash --init-f
iles/app/out/vs/workbench/contrib/terminal/common/scripts/shellIntegration-bash.sh
3549    3822   3822    3822   pts/3      3822  Ss+    1000    0:00  /usr/bin/bash --init-f
iles/app/out/vs/workbench/contrib/terminal/common/scripts/shellIntegration-bash.sh
3789    6378   6378    3789   pts/2      6378  S+     1000    0:00  ./HelloWorld-getchar
3566    6410   6410    3566   pts/1      6410  S+     1000    0:00  ./HelloWorld-getchar
3549    6421   6421    6421   pts/4      6454  Ss     1000    0:00  /usr/bin/bash --init-f
iles/app/out/vs/workbench/contrib/terminal/common/scripts/shellIntegration-bash.sh
6421    6454   6454    6421   pts/4      6454  S+     1000    0:00  ./HelloWorld-getchar
3549    6900   6900    6900   pts/5      11109 Ss     1000    0:00  /usr/bin/bash --init-f
iles/app/out/vs/workbench/contrib/terminal/common/scripts/shellIntegration-bash.sh
3822    10975  10975    3822   pts/3      3822  T      1000    0:00  top
3822    10976  10975    3822   pts/3      3822  T      1000    0:00  more
6900    11063  11063    6900   pts/5      11109 T      1000    0:00  top
6900    11064  11063    6900   pts/5      11109 T      1000    0:00  more
6900    11109  11109    6900   pts/5      11109 R+    1000    0:00  ps j
6900    11110  11109    6900   pts/5      11109 S+     1000    0:00  more
```

图 15 top | more 以及 ps j 命令查看到带有 PPID、PID、PGID、SID 字段的回显日志

从回显的详细字段中可以精准看出：**bash 本身作为会话的领头进程**，其 PID 被用作整个会话的 SID；而后台运行的复合命令（如 top 与 more）则被统一归入一个独立的后台进程组，并拥有共同的 PGID，进一步结合操作系统的作业控制与信号分发机制分析，进程组的设立为信号的批量投递提供了实体基础：

(1) 前台进程组的交互控制：前台进程组持有终端控制权。当用户在控制终端键入中断键（Ctrl-C 或 Delete）或退出键（Ctrl-\）时，内核会将生成的中断信号（SIGINT）或退出信号（SIGQUIT）精准广播至前台进程组内的所有进程；同理，挂起键（Ctrl+Z）触发的 SIGTSTP 信号也会批量下发，从而实现整个作业序列的同步启停。

(2) 后台进程组的隔离管控：加 “&” 运行的进程组处于后台，被剥夺了控制终端的权限。若后台进程违规尝试读取或写入控制终端，内核将直接派发 SIGTTIN 或 SIGTTOU 信号将其强制挂起，避免终端输入输出发生混乱。

(3) 会话层的级联终止：当检测到终端 modem 或网络断线时，挂断信号（SIGHUP）会首先发送给会话领头进程（bash）。当 bash 终止后，内核会随之将该信号送往当前的前台进程组，触发进程组的默认终止动作。这种 “会话统筹终端、进程组隔离作业、统一批量派发信号” 的嵌套模型，极大简化了 Shell 对多并发任务的协调与管理。

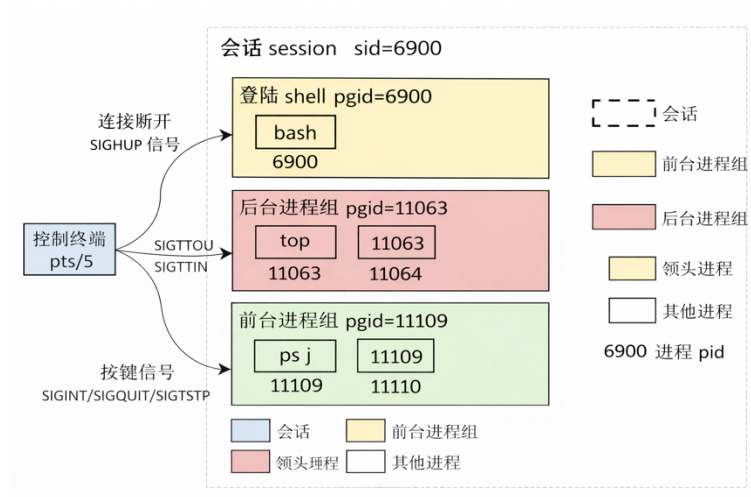


图 16 会话、进程组以及前后台关系例子

2.1.3 基于 Shell 前端命令的进程调度与控制机制

在深入分析底层 C 语言系统调用之前，本环节首先验证基于 Shell 前端的进程级管控机制

(1) 基于 Shell 脚本的进程自动化创建与权限模型：除了在终端直接交互式键入命令，Linux 支持通过 Shell 脚本以批量化、自动化的方式调度新进程的创建，实验中编写了一个简易的 Bash 脚本，脚本内部嵌套调用了外部系统命令（echo）与用户级可执行程序（HelloWorld）。

核心脚本代码如下：

```
#!/usr/bin/bash

echo Running a shell script!

./HelloWorld
```

实验重点验证了 Linux 文件系统的执行权限管控：**新创建的脚本文件默认仅有读写权限**（通过 `ls -l` 观测），直接执行将被内核拒绝；必须显式使用 **`chmod a+x` 命令向文件赋权**（增加 x 属性）后，Shell 方能正确解析脚本头部的解释器路径，并依次 fork 出子进程来完成任务。

```
• liwenjun@liwenjun-VMware-Virtual-Platform:~/OS/Integrate_1$ vi hello.sh
• liwenjun@liwenjun-VMware-Virtual-Platform:~/OS/Integrate_1$ ls -l hello.sh
-rw-rw-r-- 1 liwenjun liwenjun 59  4月 28 20:50 hello.sh
• liwenjun@liwenjun-VMware-Virtual-Platform:~/OS/Integrate_1$ chmod a+x hello.sh
• liwenjun@liwenjun-VMware-Virtual-Platform:~/OS/Integrate_1$ ls -l hello.sh
-rwxrwxr-x 1 liwenjun liwenjun 59  4月 28 20:50 hello.sh
• liwenjun@liwenjun-VMware-Virtual-Platform:~/OS/Integrate_1$ bash hello.sh
Running a shell script!
HelloWorld!
Please press 'Enter' to exit...
```

图 17 ls -l 查看脚本初始权限、chmod a+x 赋权操作以及脚本成功运行输出的回显日志

(2) 基于信号分发(kill)的进程定向撤销控制：在终端通过命令行撤销目标进程，其底层本质是系统调用将控制信号投递至指定的 PCB，实验中首先在后台运行长时间阻塞的程序（如 HelloWorld-getchar），通过 “`ps -ef | grep HelloWorld-getchar`” 这一经典的进程嗅探命令组合，精准捕获目标进程的实体 PID。

获取 PID 后，通过执行 “`kill <PID>`” 命令（默认发送 SIGTERM 即 15 号软性终止信号），成功使得该程序终止。对于无法响应常规撤销信号的进程，系统还提供了**最强力且不可被屏蔽或忽略的 “`kill -9 <PID>`”**

(SIGKILL 信号) 来进行强制内存回收。此外, killall 命令则展示了直接基于“进程映像名称”进行批量信号广播的高级用法。

```
• Liwenjun@Liwenjun-VMware-Virtual-Platform:~/OS/Integrate_1$ ps -ef | grep HelloWorld
liwenjun  13796  13768  0 20:54 pts/6    00:00:00 ./HelloWorld
liwenjun  13809  3822  0 20:54 pts/3    00:00:00 grep --color=auto HelloWorld
• Liwenjun@Liwenjun-VMware-Virtual-Platform:~/OS/Integrate_1$ kill 13796
• Liwenjun@Liwenjun-VMware-Virtual-Platform:~/OS/Integrate_1$ ps -ef | grep HelloWorld
liwenjun  13938  3822  0 20:54 pts/3    00:00:00 grep --color=auto HelloWorld
```

图 18 通过 ps -ef 精确查找到 HelloWorld-getchar 的 PID, 并执行 kill 命令将其成功撤销的回显日志

2.2 进程的创建与状态控制

2.2.1 基于 fork() 创建多级嵌套进程

通过编写 C 语言代码直接调用 unistd.h 中的 fork() 系统调用, fork() 执行后会完整复制父进程的 PCB, 导致执行流在此产生分叉, 在 fork-twice-demo 测试程序中, 父进程率先 fork 出子进程 A, 随后父进程与子进程 A 各自向下执行又分别触发一次 fork(), 最终在内存中准确裂变出 1 个父进程、2 个子进程以及 1 个孙辈进程。

核心逻辑代码片段如下:

```
// 连续执行两次fork不加阻断, 验证进程树裂变
int pid = fork();
if(pid == 0) {
    printf("This is the child process!\n");
    fork(); // 子进程再次分叉产生孙进程
} else if(pid > 0) {
    printf("This is the parent process!\n");
    fork(); // 父进程再次分叉产生第二个子进程
}
```

图 19 多级嵌套核心代码

运行之后通过 pstree 查看到四节点多层次进程树:

```
• Liwenjun@Liwenjun-VMware-Virtual-Platform:~/OS/Integrate_1$ ./fork-twice-demo
This is the child process!
This is the parent process! child process id = 63756
```

图 20 fork-demo 的输出

```
• Liwenjun@Liwenjun-VMware-Virtual-Platform:~/OS/Integrate_1$ ps j | grep fork
6900  63755  63755  6900 pts/5    63755 S+  1000  0:00 ./fork-twice-demo
63755  63756  63755  6900 pts/5    63755 S+  1000  0:00 ./fork-twice-demo
63756  63757  63755  6900 pts/5    63755 S+  1000  0:00 ./fork-twice-demo
63755  63758  63755  6900 pts/5    63755 S+  1000  0:00 ./fork-twice-demo
13768  63844  63843  13768 pts/6    63843 S+  1000  0:00 grep --color=auto fork
• Liwenjun@Liwenjun-VMware-Virtual-Platform:~/OS/Integrate_1$ pstree 63755
fork-twice-demo├─fork-twice-demo─fork-twice-demo
               └─fork-twice-demo
```

图 21 ps 和 pstree 回显日志

2.2.2 复现僵尸进程 (Zombie) 并解析机理

僵尸进程产生的根源在于子进程先于父进程退出, 但父进程未执行资源回收。实验中设计子进程打印信息后立刻调用 exit(0) 销毁其实体资源, 但强制让父进程 sleep(10) 挂起而不调用 wait()。在这 10 秒的真空期内, 子进程的 PCB (task_struct) 及内核栈被迫保留在系统中供父进程日后读取退出状态, 此时用 ps 工具探测, 准确捕获到子进程状态异变为 Z+ (<defunct>)。

核心逻辑代码片段如下:

```

if(pid == 0) {
    exit(0); // 子进程主动退出，资源大部分释放但PCB残留
} else {
    sleep(10); // 父进程休眠，故意推迟调用wait()回收PCB
    wait(NULL); // 10秒后醒来打扫战场，僵尸进程彻底消失
}

```

图 22 僵尸进程核心代码

```

• liwenjun@liwenjun-VMware-Virtual-Platform:~/OS/Integrate_1$ ./zombie-demo
Hi father! I'm a ZOMBIE
○ liwenjun@liwenjun-VMware-Virtual-Platform:~/OS/Integrate_1$ █

```

图 23 fork-demo 的输出

```

• liwenjun@liwenjun-VMware-Virtual-Platform:~/OS/Integrate_1$ ps j | grep zombie
6900  64259  64259  6900 pts/5      64259 S+   1000  0:00 ./zombie-demo
64259  64260  64259  6900 pts/5      64259 Z+   1000  0:00 [zombie-demo] <defunct>
13768  64327  64326  13768 pts/6      64326 S+   1000  0:00 grep --color=auto zombie

```

图 24 ps 和 pstree 回显日志

2.2.3 再现孤儿进程并解析内核收养机理

孤儿进程产生的场景与僵尸进程恰好相反：当父进程先于子进程结束退出时，仍在运行的子进程便会成为“孤儿进程”。为了避免这些失去父进程的子进程在最终结束时因无人回收 PCB 而沦为僵尸进程，Linux 内核引入了极其严密的“孤儿收养”机制。当内核在销毁父进程时，会动态扫描其挂载的子进程链表，并将其名下所有存活的孩子进程强行“过继”给系统的 1 号进程（即 init 进程或现代 Linux 中的 systemd），此后，由 1 号进程充当它们的父进程，并在它们结束时自动调用 wait() 回收残余的内核数据结构，因此孤儿进程本身绝不会造成系统内存资源的泄漏。

为验证这一底层收养机制，实验中编写了自动化的 orphan-demo 程序。程序利用 sleep() 函数制造严格的时序差：使父进程在 1 秒后立刻执行 exit(0) 销毁实体，而子进程则被挂起休眠，强行滞留在系统中成为孤儿：

```

liwenjun@liwenjun-VMware-Virtual-Platform:~/OS/Integrate_1$ ./fork-demo
[Parent] My PID is 4435, I will exit in 1 second...
[Child] My PID is 4436, my original PPID is 4435
liwenjun@liwenjun-VMware-Virtual-Platform:~/OS/Integrate_1$ [Child] I am an orphan now! My new PPID is 1

```

图 25 孤儿进程的示例 1

在传统的 Unix/Linux 系统中（如老版本的 SysV init），孤儿进程会被直接“过继”给系统的 1 号进程（init 进程），在本实验的现代 Linux 运行环境中（基于 systemd），内核引入了“亚收养进程（Subreaper）”特性（通过 prctl(PR_SET_CHILD_SUBREAPER) 系统调用实现）。因此，实验中观察到子进程的 PPID 并未变为 1，而是跃迁变为了当前用户态会话的 systemd --user 实例进程号（或桌面环境的守护进程 PID）。

```

• liwenjun@liwenjun-VMware-Virtual-Platform:~/OS/Integrate_1$ [Child] I am an orphan now! My new PPID is 1508
./fork-demo
[Child] My PID is 4177, my original PPID is 4176
[Parent] My PID is 4176, I will exit in 1 second...
○ liwenjun@liwenjun-VMware-Virtual-Platform:~/OS/Integrate_1$ [Child] I am an orphan now! My new PPID is 1508

```

图 26 孤儿进程的示例 2

这一关键数据突变不仅完美实证了内核的孤儿收养防泄漏机制，更验证了现代 Linux 操作系统在用户级会话进程管理上的架构演进。最后，当子进程生命周期结束时，由其新的 Subreaper 父进程自动调用 wait() 回收了残留的内核数据结构，全程未产生任何系统资源浪费。

2.2.4 基于 exec 函数族完成进程影像变身

验证 Linux 下“进程创建分离模型”，fork() 仅负责复制躯壳，而 execve() 负责注入灵魂，实验中子进程调用

execve()将自身的内存影像（包括代码段、数据段、堆栈等）彻底替换为/usr/bin/ls 程序的影像。

实验结果明确显示，原本紧跟在 execve()之后的 printf()语句被彻底丢弃未执行，终端输出的变为了当前目录下的文件列表，完美证实了影像覆盖机制。

核心逻辑代码片段如下：

```
char *argv_new[ ]={"ls", "-al", "/etc/passwd", NULL};
char *envp_new[ ]={"PATH=/usr/bin", NULL};
// 彻底覆盖当前进程上下文，变身为ls命令
execve("/usr/bin/ls", argv_new, envp_new);
printf("此句由于影像已被替换，绝对不会被执行到!\n");
```

图 27 fork-exec-demo.c

```
● liwenjun@liwenjun-VMware-Virtual-Platform:~/OS/Integrate_1$ ./fork-exec-demo
This is the parent process! child process id = 5203
This is the child process!
-rw-r--r-- 1 root root 3080 May  9 2025 /etc/passwd
```

图 28 fork-exec-demo 成功变身为 ls 命令并输出 passwd 文件信息的回显日志

2.3 线程概念与多线程并发机制验证

2.3.1 进程与线程的内核级资源隔离与共享模型剖析

在引入线程概念前，传统的进程既是资源分配的基本单位，又是调度执行的基本单位。在多并发场景下，若采用多进程模型，系统需要为每个执行流分配独立的虚拟内存空间与系统资源，导致极大的时空开销，为打破这一瓶颈，操作系统引入了线程机制，将“资源分配”与“调度执行”两种属性进行了解耦：

(1) 执行流的独立性（调度单位）：在统一进程实体内，可以并发存在多个指令执行流（线程）。为了保证各执行流在 CPU 调度中断时能正确保留现场，Linux 内核为每个线程分配了独立的**进程控制块（task_struct）**，其中记录了各自独立的**指令指针（IP）**、CPU 寄存器上下文以及用于追溯函数调用链的独立用户态堆栈（Stack）。

(2) 资源的共享性（资源单位）：在 Linux 底层模型中，同一进程下的多个线程（如 P2-0 与 P2-1）尽管拥有独立的 task_struct，但其内部的内存描述符指针（mm 成员）均指向同一个 mm_struct 结构体。这意味着它们完全共享同一套虚拟地址空间（包括代码段、全局数据段）及文件描述符表，从而避免了内存空间分配的巨大开销。

此外，即使是基于同一可执行文件创建的不同进程（如 P1 与 P2），虽然拥有独立的虚存空间与 mm_struct，但现代 OS 依然通过页表映射机制让它们在底层共享了相同的物理代码段（Code Segment），仅对可写数据区（Data Segment）进行隔离映射。

2.3.2 基于 pthread 的轻量级进程（LWP）机制验证

基于上述理论模型，本实验引入 pthread_create()在同一进程实体内开辟多个独立的指令执行流，以实证 Linux 内核中的线程调度行为。在 C 语言代码中，主程序拉起独立线程，并在各自的执行流中调用 sleep()产生并发。编译时显式链接-lpthread 库。。

```
● liwenjun@liwenjun-VMware-Virtual-Platform:~/OS/Integrate_1$ ./pthread-demo
This is the main process.
This is a pthread.
● liwenjun@liwenjun-VMware-Virtual-Platform:~/OS/Integrate_1$
```

图 29 pthread-demo 的输出

```

● liwenjun@liwenjun-VMware-Virtual-Platform:~/05/Integrate_1$ ps -aLf
UID      PID     PPID    LWP   C  NLWP  STIME  TTY          TIME CMD
liwenjun 1580    1572    1580   0   4 23:01 tty2      00:00:00 /usr/libexec/gnome-session-binary --session=ubuntu
liwenjun 1580    1572    1622   0   4 23:01 tty2      00:00:00 /usr/libexec/gnome-session-binary --session=ubuntu
liwenjun 1580    1572    1623   0   4 23:01 tty2      00:00:00 /usr/libexec/gnome-session-binary --session=ubuntu
liwenjun 1580    1572    1624   0   4 23:01 tty2      00:00:00 /usr/libexec/gnome-session-binary --session=ubuntu
liwenjun 4330    4244    4330   0   1 23:11 tty3      00:00:00 -bash
liwenjun 5647    3770    5647   0   2 23:43 pts/2      00:00:00 ./pthread-demo
liwenjun 5647    3770    5648   0   2 23:43 pts/2      00:00:00 ./pthread-demo
liwenjun 5665    4033    5665  99   1 23:43 pts/3      00:00:00 ps -aLf

```

图 30 pthread-demo 运行时执行 ps -aLf 的输出

通过“ps -aLf”命令监控进程与线程状态，从回显数据中可以明确观测到：主线程与新创建的子线程拥有绝对相同的进程号（PID 列一致），说明它们被封装在同一个资源容器（mm_struct）内；但 Linux 内核为它们分配了不同的轻量级进程号（LWP 列），且代表统一进程实体内线程总数的 NLWP 列明确显示为 2。这一现象完美呼应了 2.3.1 节中的内核模型理论：在 Linux 底层，线程作为独立调度的实体（持有各自独立的 task_struct 与 LWP 标识），寄生于共享的进程资源空间内高效运行。

2.4 进程与线程内核资源开销深度对比

本环节是理解进程与线程本质区别的核心，通过分别批量生成 100 个子进程与 100 个子线程，并实时提取系统 slab 分配器的统计数据，获得了极具说服力的资源消耗对比账单

2.4.1 内核控制块(PCB)开销平行对比

无论是进程还是线程，只要是能够被 CPU 独立调度的执行流，Linux 都必须为其分配核心控制块。观测数据表明，无论是运行 fork-100-demo 还是 pthread-100-demo，/proc/slabinfo 中的 task_struct 对象数量均发生了约 100 个的同等增量，证明在任务调度维度的开销二者是平级的。

```

● liwenjun@liwenjun-VMware-Virtual-Platform:~/05/Integrate_1$ sudo cat /proc/slabinfo | grep task_struct
task_struct 1028 1062 10688 3 8 : tunables 0 0 0 : slabdata 354 354 0
● liwenjun@liwenjun-VMware-Virtual-Platform:~/05/Integrate_1$ sudo cat /proc/slabinfo | grep task_struct
task_struct 1109 1116 10688 3 8 : tunables 0 0 0 : slabdata 372 372 0

```

图 31 fork-100-demo 程序运行前后，cat /proc/slabinfo | grep task_struct 分别激增约 100 个对象的回显日志

```

● liwenjun@liwenjun-VMware-Virtual-Platform:~/05/Integrate_1$ sudo cat /proc/slabinfo | grep task_struct
task_struct 1017 1050 10688 3 8 : tunables 0 0 0 : slabdata 350 350 0
● liwenjun@liwenjun-VMware-Virtual-Platform:~/05/Integrate_1$ sudo cat /proc/slabinfo | grep task_struct
task_struct 1116 1116 10688 3 8 : tunables 0 0 0 : slabdata 372 372 0

```

图 32 pthread-100-demo 程序运行前后，cat /proc/slabinfo | grep task_struct 分别激增约 100 个对象的回显日志

2.4.2 内存描述符与虚拟内存区域开销差异对比

这是二者产生“重量级”与“轻量级”分水岭的核心论据。

- (1) 首先运行 fork-100-demo，数据表明系统新增了约 80 个 mm_struct（代表分配了 80 套独立的虚拟内存页表体系），并且 vm_area_struct 激增了约 2300 个（因为每个新进程都需要为自身的代码段、数据段、堆栈等独立划分虚拟内存区）。
- (2) 随后运行 pthread-100-demo，惊人的差异出现：mm_struct 的数据增量几乎为 0（证明 100 个线程完全寄生并复用了主进程的内存地址空间，未发生任何大件拷贝），而 vm_area_struct 仅仅新增了约 240 个（这是因为每个子线程在共享地址空间内，仅仅需要额外划分一块专属的独立用户态栈区），无可辩驳地证实了线程资源消耗极小的理论依据。

```

● liwenjun@liwenjun-VMware-Virtual-Platform:~/05/Integrate_1$ sudo cat /proc/slabinfo | grep mm_struct
mm_struct 214 252 1536 21 8 : tunables 0 0 0 : slabdata 12 12 0
● liwenjun@liwenjun-VMware-Virtual-Platform:~/05/Integrate_1$ sudo cat /proc/slabinfo | grep mm_struct
mm_struct 294 294 1536 21 8 : tunables 0 0 0 : slabdata 14 14 0

```

图 33 fork-100-demo 运行前后的/proc/slabinfo 中关于 mm_struct 数量


```

• Liwenjun@liwenjun-VMware-Virtual-Platform:~/OS/Integrate_1$ sudo cat /proc/slabinfo | grep vm_area_struct
vm_area_struct 43135 43197 192 21 1 : tunables 0 0 0 : slabdata 2057 2057 0
• Liwenjun@liwenjun-VMware-Virtual-Platform:~/OS/Integrate_1$ sudo cat /proc/slabinfo | grep vm_area_struct
vm_area_struct 45486 45486 192 21 1 : tunables 0 0 0 : slabdata 2166 2166 0

```

图 34 fork-100-demo 运行前后的/proc/slabinfo 中关于 vm_area_struct 数量

```

• Liwenjun@liwenjun-VMware-Virtual-Platform:~/OS/Integrate_1$ sudo cat /proc/slabinfo | grep mm_struct
mm_struct 224 273 1536 21 8 : tunables 0 0 0 : slabdata 13 13 0
• Liwenjun@liwenjun-VMware-Virtual-Platform:~/OS/Integrate_1$ sudo cat /proc/slabinfo | grep mm_struct
mm_struct 224 273 1536 21 8 : tunables 0 0 0 : slabdata 13 13 0

```

图 35 pthread-100-demo 运行前后的/proc/slabinfo 中关于 mm_struct 数量

```

• Liwenjun@liwenjun-VMware-Virtual-Platform:~/OS/Integrate_1$ sudo cat /proc/slabinfo | grep vm_area_struct
vm_area_struct 42744 43239 192 21 1 : tunables 0 0 0 : slabdata 2059 2059 0
• Liwenjun@liwenjun-VMware-Virtual-Platform:~/OS/Integrate_1$ sudo cat /proc/slabinfo | grep vm_area_struct
vm_area_struct 42987 43239 192 21 1 : tunables 0 0 0 : slabdata 2059 2059 0

```

图 36 fork-100-demo 运行前后的/proc/slabinfo 中关于 vm_area_struct 数量

3. 预备 2-进程间通信与同步

3.1 进程间通信机制验证

3.1.1 无名管道与标准输入输出重定向原理验证

广义上的进程间通信涵盖了磁盘文件与网络传输，而狭义上的传统 IPC 机制主要依赖内核管理的内存缓冲区。

无名管道 (Pipe) 是所有 Unix 系统提供的一种**单向信道**，其本质是**内核维护的一个缓冲区**。在 Shell 处理类似 “ls | more” 的管道命令时，其底层原理是首先调用 pipe() 创建管道并返回**一对读写文件描述符**，通过两个文件描述符（整数）来指代管道缓冲区的读端和写端，随后两次调用 fork() 创建两个子进程，并通过 dup2() 系统调用将子进程 1 的标准输出（描述符 1）重定向到管道写端，将子进程 2 的标准输入（描述符 0）重定向到管道读端，最后调用 execve() 执行外部程序，从而完成数据的单向流动。

实验中编写了 pipe-demo 程序实证了上述理论：父进程在 fork() 前创建管道，随后父进程**主动关闭读端**(fds[0]) 并通过 write() 向写端发出格式化字符串，子进程**主动关闭写端** (fds[1]) 并通过 read() 从读端成功接收到信息并打印，完美验证了无名管道依赖文件描述符继承的父子进程通信机制。

父子进程都有读写权限，为什么要主动关闭呢？

(1) **读端永远等不到 EOF 导致写端死锁**：fork() 之后，父子进程都拥有 fd[0](读) 和 fd[1](写)，此时，管道的写端口总引用计数为 2，内核机制是当一个进程调用 read() 读管道时，如果管道里没数据了，它是直接返回 0（表示文件结束 EOF），还是原地阻塞死等？**关键看写端口的引用计数**！只有当管道的所有写端都被关闭（引用计数为 0）时，read() 才会收到 EOF 并返回 0，告诉子进程“对面写完了”，此时如果子进程不把自己的 fd[1] 关闭，那么即使父进程写完数据并退出了（父进程的 fd[1] 关了），内核一看，**管道的写端口引用计数还是 1**（因为子进程自己手里还捏着一个写描述符）。内核会想：“既然还有人可能往里写东西，那读进程就继续等着吧！”于是，子进程的 read() 会永远阻塞卡死在那里，程序永远无法正常结束

(2) **写端无法感知读者死亡 导致写端死锁或资源浪费**：如果读端进程突然崩溃或退出了，写端还在往管道里写数据，为了防止这种浪费，Linux 规定：如果管道的所有读端都被关闭了，谁还敢往里 write()，内核就会直接给它发送一个 SIGPIPE 信号（默认把写进程杀死），或者让 write() 返回 -1 报错；所以如果父进程不把自己的读端 fd[0] 关闭。当子进程意外死亡后，内核一看，管道的读端引用计数还是 1（因为父进程自己手里还捏着一个读描述符）。内核觉得“还有人能读”，就不会发送 SIGPIPE。于是，父进

程会继续往管道里疯狂写数据，直到把内核 64KB 的管道缓冲区全部塞满，然后父进程的 write() 也会被永久阻塞卡死

核心逻辑代码片段如下：

```
// 创建无名管道并产生分支
pipe(fds);
pid = fork();
if(pid == 0) {
    close(fds[1]); // 子进程关闭写端
    nwr = read(fds[0], buf, sizeof(buf)); // 阻塞读取数据
} else {
    close(fds[0]); // 父进程关闭读端
    nwr = write(fds[1], buf, sizeof(buf)); // 写入数据流
}
```

图 37 pipe-demo.c

```
wenjun@2023150001:~/intergrated_exp1$ ./pipe-demo
This is parent process, pid = 1977
Parent:sending message...
Parent:send 128 bytes to child.
This is child process, pid = 1978
Child:waiting for message...
Child:received "Message from parent!"
wenjun@2023150001:~/intergrated_exp1$
```

图 38 父进程发送数据，子进程成功接收数据的输出画面

3.1.2 命名管道（FIFO）的阻塞与唤醒机制验证

无名管道受限于只能在具有血缘关系的进程间使用，命名管道（FIFO）突破了此限制，通过在文件系统目录树中建立拥有可访问磁盘索引节点（inode）的特殊文件，实现了任意进程间的通信。

实验中通过“mkfifo os-exp-fifo”命令创建了命名管道，并通过 ls -l 观察到其文件类型标识为“p”，在终端 A 中使用 cat 尝试读取该管道文件时，由于此时管道内无数据写入，cat 进程自动陷入了内核态的阻塞休眠状态。随后在独立的终端 B 中执行“echo Hello, Named PIPE! > os-exp-fifo”向管道写入数据，内核在检测到数据后立即唤醒了终端 A 中的 cat 进程，并成功将缓冲数据回显至屏幕终端。这不仅验证了 FIFO 的跨进程通信能力，也直观展现了管道“空则阻塞读、满则阻塞写”的内核同步特性。

```
wenjun@2023150001:~/intergrated_exp1$ ./pipe-demo
This is parent process, pid = 1977
Parent:sending message...
Parent:send 128 bytes to child.
This is child process, pid = 1978
Child:waiting for message...
Child:received "Message from parent!"
wenjun@2023150001:~/intergrated_exp1$ mkfifo os-exp-fifo
wenjun@2023150001:~/intergrated_exp1$ ls -l os-exp-fifo
prw-rw-r-- 1 wenjun wenjun 0 May  5 16:04 os-exp-fifo
wenjun@2023150001:~/intergrated_exp1$ cat os-exp-fifo
```

图 39 终端 A 的 cat 命令陷入阻塞

```
wenjun@2023150001:~/intergrated_exp1$ ./pipe-demo
This is parent process, pid = 1977
Parent:sending message...
Parent:send 128 bytes to child.
This is child process, pid = 1978
Child:waiting for message...
Child:received "Message from parent!"
wenjun@2023150001:~/intergrated_exp1$ mkfifo os-exp-fifo
wenjun@2023150001:~/intergrated_exp1$ ls -l os-exp-fifo
prw-rw-r-- 1 wenjun wenjun 0 May  5 16:04 os-exp-fifo
wenjun@2023150001:~/intergrated_exp1$ cat os-exp-fifo
Hello, Named PIPE!
wenjun@2023150001:~/intergrated_exp1$
```

图 40 终端 B 利用 echo 写入数据后终端 A 被唤醒并打印字符串

3.1.3 System V IPC：消息队列的管理与通信

Linux 底层的 System V IPC 包含**信号量集**、**共享内存**和**消息队列**，它们均通过系统唯一的 IPC 标识符（ID）进行寻址定位，且内核会根据 **IPC key** 来进行 ID 的转换。

在实验前通过 `ipcs -ql` 命令观察到了系统对消息队列的数量与字节等容量限制参数。

```
wenjun@2023150001:~/intergrated_exp1$ ipcs
----- Message Queues -----
key      msqid      owner      perms      used-bytes   messages
----- Shared Memory Segments -----
key      shmid      owner      perms      bytes       nattch     status
----- Semaphore Arrays -----
key      semid      owner      perms      nsems
```

图 41 ipcs 命令的输出

```
wenjun@2023150001:~/intergrated_exp1$ ipcs -ql
----- Messages Limits -----
max queues system wide = 32000
max size of message (bytes) = 8192
default max size of queue (bytes) = 16384
```

图 42 ipcs -ql 命令的输出

实验中利用基于 `msgget()` 等接口编写的 `msgtool` 程序进行了全流程验证：程序首先利用 `ftok()` 根据**文件路径与字符**生成唯一的 `key`，进而传入 `msgget(key, IPC_CREAT|0660)` 系统调用在内核中**开辟消息队列**，`IPC_CREAT` 表示消息队列若是还不存在则创建一个新的、`0660` 表示创建者及其同组用户可以读（收）也可以写（发）消息。通过执行**发信操作（命令行参数 s）**将携带有特定类型（type）标签与内容的结构体数据打入队列，随后执行 `ipcs -q` 清晰观测到队列 ID 生成且**已用字节数变为 20、消息数量变为 1**。

```
wenjun@2023150001:~/intergrated_exp1$ ./msgtool s 1 "Hello,my_msg_queue!"
Sending a message
wenjun@2023150001:~/intergrated_exp1$ ipcs -q
----- Message Queues -----
key      msqid      owner      perms      used-bytes   messages
0x6d001019 0          wenjun      660         20            1
```

图 43 发送消息、ipcs -q 查看到队列

启动独立的读进程执行收信操作（**命令行参数 r**），成功通过指定的 `type` 字段将对应的消息出队。读取完毕后，再次使用 `ipcs -q` 查验，发现队列内消息已被内核自动清理并恢复为空。

```
wenjun@2023150001:~/intergrated_exp1$ ./msgtool r 1
Reading a message
Type: 1 Text: Hello,my_msg_queue!
wenjun@2023150001:~/intergrated_exp1$ ipcs -q
----- Message Queues -----
key      msqid      owner      perms      used-bytes   messages
0x6d001019 0          wenjun      660         0             0
wenjun@2023150001:~/intergrated_exp1$ █
```

图 44 读取消息后再次 ipcs -q 查看到消息清空

核心逻辑代码片段如下：

图 45 msgtool.c

共享内存是 System V IPC 中效率最高的数据交互方式，其核心原理是**内核划拨一段物理内存，并允许不同的进程将其直接映射到各自独立的虚拟地址空间中。**

```
wenjun@2023150001:~/intergrated_expl$ ./shmget-demo
Successfully created segment : 0

----- Shared Memory Segments -----
key          shmid      owner      perms      bytes      nattch     status
0x00000000   0          wenjun     666        4096       0          
```

图 46 创建共享内存

```

● wenjun@2023150001:~/intergrated_expl$ ./shmatt-write-demo 0
segment attached at 0xffff7fb06000

----- Shared Memory Segments -----
key      shmid  owner  perms  bytes  nattch  status
0x00000000 0      wenjun 666    4096   1

Press Enter to detach...

segment detached

----- Shared Memory Segments -----
key      shmid  owner  perms  bytes  nattch  status
0x00000000 0      wenjun 666    4096   0

```

图 47 写入数据观测 nattch 变化

```

jen@jun2023150001:~/integrated_exp1$ ps -a
  PID TTY          TIME CMD
  116   tty1      00:00:00 bash
    282   pts/2      00:00:00 sshmt-write-de
    285   pts/1      00:00:00 ps
jen@jun2023150001:~/integrated_exp1$ cat /proc/2825/maps
aaaa1d58000-aaaa1d59000 r-xp 00000000 fd:00 683269 /home/wenjun/integrated_exp1/sshmt-write-demo
aaaa1d68000-aaaa1d69000 r-p 00000000 fd:00 683269 /home/wenjun/integrated_exp1/sshmt-write-demo
aaaa1d69000-aaaa1d6a000 r-wp 00001000 fd:00 683269 /home/wenjun/integrated_exp1/sshmt-write-demo
aaaa58ee000-aaaa58f0000 r-wp 00000000 00:00 0 /heap
ffffbf32a000-ffffbf485000 r-xp 00000000 fd:00 133973 /usr/lib/arch64-linux-gnu/libc-2.31.so
ffffbf485000-ffffbf494000 r-p 00158000 fd:00 133973 /usr/lib/arch64-linux-gnu/libc-2.31.so
ffffbf494000-ffffbf498000 r-p 00158000 fd:00 133973 /usr/lib/arch64-linux-gnu/libc-2.31.so
ffffbf498000-ffffbf49a000 r-wp 0015e000 fd:00 133973 /usr/lib/arch64-linux-gnu/libc-2.31.so
ffffbf49a000-ffffbf49c000 r-p 00000000 00:00 0 /usr/lib/arch64-linux-gnu/ld-2.31.so
ffffbf49c000-ffffbf4be000 r-xp 00000000 fd:00 132369 /usr/lib/arch64-linux-gnu/ld-2.31.so
ffffbf4c000-ffffbf4c5000 r-wp 00000000 00:00 0 /var/
ffffbf4c000-ffffbf4c000 r-ws 00000000 00:01 0 /SYSV00000000 (deleted)
ffffbf4cc000-ffffbf4cd000 r-p 00000000 00:00 0 [var]
ffffbf4cd000-ffffbf4ce000 r-xp 00000000 00:00 0 [vdso]
ffffbf4ce000-ffffbf4cf000 r-p 00021000 fd:00 132369 /usr/lib/arch64-linux-gnu/ld-2.31.so
ffffbf4cf000-ffffbf4d0000 r-p 00022000 fd:00 132369 /usr/lib/arch64-linux-gnu/ld-2.31.so
ffffbf235000-ffffbf235000 r-p 00000000 00:00 0 [stack]
jen@jun2023150001:~/integrated_exp1$

```

图 48 shmatt-write-demo 映射共享内存时的进程布局

随后启动毫不相干的 `shmatt-read-demo` 进程并再次执行映射操作时, 依旧完美读取到了前一个进程写下的 “Hello shared memory!” 字符串, 证实了只要未被明确调用 `ipcrm` 或代码级销毁, 共享内存中的数据在系统

存续期间将始终留存。

```
wenjun@2023150001:~/intergrated_exp1$ ./shmatt-read-demo 0
segment attached at 0xffffb3559000
The string in SHM is :Hello shared memory!
wenjun@2023150001:~/intergrated_exp1$
```

图 49 读取消息后再次 ipcs -q 查看到消息清空

核心逻辑代码片段如下：

```
// 申请物理内存，并映射至进程虚拟地址空间
shm_id = shmget(IPC_PRIVATE, BUFSZ, 0666);
shm_buf = shmat(shm_id, 0, 0);
strcpy(shm_buf, "Hello shared memory!\n"); // 直接通过指针发生跨进程写入
// 读写完毕后解除当前进程的映射
shmdt(shm_buf);
```

图 50 shmget-demo.c

3.2. 进程/线程间同步机制验证

3.2.1.POSIX 有名信号量的 P/V 操作与阻塞唤醒验证

进程间的通信解决了数据流转问题，而执行顺序的先后互斥逻辑则依赖同步机制。POSIX 信号量相较于 System V 更为简洁，其分为依赖文件系统路径名可跨进程使用的有名信号量，以及仅在进程内存中针对线程的无名信号量。

信号量核心函数如下：

```
// 获取有名信号量并执行原子级的减操作（可能会引发阻塞挂起）
sem = sem_open(argv[1], 0);
sem_wait(sem);
// 另一进程执行增操作以唤醒挂起的进程
sem_post(sem);
// 撤销指定的信号量
sem_unlink(argv[1]);
```

图 51 信号量核心函数

实验中通过 psem-named-open 程序调用 `sem_open()` 及 `O_CREAT` 标志，在系统中初始化了一个数值为 1 的有名信号量：

```
wenjun@2023150001:~/intergrated_exp1$ ./psem-named-open-demo /mysem
Semaphore /mysem created successfully.
wenjun@2023150001:~/intergrated_exp1$
```

图 52 终端 A（创建信号量）

接着在终端执行 psem-named-wait-demo 模拟 V 操作（资源减 1，`sem_wait`），第一次执行时信号量由 1 归零程序顺利通过：

```
wenjun@2023150001:~/intergrated_exp1$ ./psem-named-wait-demo /mysem
Ready to wait (P operation)...
pid 3151 has semaphore, value=0
wenjun@2023150001:~/intergrated_exp1$
```

图 53 终端 A（第一次消耗）

紧接着在同一终端内第二次运行该程序，因可用信号量已为 0，内核立即将该进程从执行态剥夺并打入阻塞休眠队列。此时在独立终端用 `ps aux` 探测，发现该进程状态确切变为了 S+：

```
wenjun@2023150001:~/intergrated_exp1$ ./psem-named-wait-demo /mysem
Ready to wait (P operation)...
```

图 54 终端 A (第二次消耗, 引发阻塞)

```
wenjun@2023150001:~/intergrated_exp1$ ps aux | grep mysem
wenjun    3178  0.0  0.0  2128  500 pts/1    S+   17:23   0:00 ./psem-named-wait-demo /mysem
wenjun    3260  0.0  0.0  5836  644 pts/2    S+   17:23   0:00 grep --color=auto mysem
wenjun@2023150001:~/intergrated_exp1$
```

图 55 终端 B (观察阻塞状态)

为了打破僵局, 在另一个终端执行 psem-named-post-demo 模拟 P 操作 (资源加 1, sem_post), 系统内核随之将挂起阻塞的 wait 进程重新唤醒, 使其完成剩余输出:

```
wenjun@2023150001:~/intergrated_exp1$ ./psem-named-post-demo /mysem
Post success! current value=1
wenjun@2023150001:~/intergrated_exp1$
```

图 56 终端 B (观察阻塞状态)

最后通过 sem_unlink()成功将系统中的信号量句柄注销, 彻底完成了一个经典的阻塞与唤醒调度闭环:

```
wenjun@2023150001:~/intergrated_exp1$ ./psem-named-unlink-demo /mysem
Semaphore /mysem unlinked.
wenjun@2023150001:~/intergrated_exp1$
```

图 57 终端 A (销毁清理)

3.2.2.多线程并发的数据竞争与互斥量保护

互斥量 (Mutex) 本质是仅用于互斥访问的退化版信号量 (二值信号量), 常用于解决多线程间由于调度不确定性带来的共享变量脏写问题。

在对照实验中, 首先运行未加锁保护的 no-mutex-demo 程序, 该程序将 256MB 的缓冲区分片派发给 16 个并发线程去统计值为 3 的元素个数, 在计数累加 (count++) 这一典型的“读取-修改-写回”非原子操作期间, 由于缺乏硬件级别的内存栅栏与系统级的互斥机制, 不同线程频繁在临界区内相互覆盖寄存器结果。连续三次运行该程序, 得出了 11268431、11302571 等完全不确定且错误的最终结果。

```
wenjun@2023150001:~/intergrated_exp1$ ./no-mutex-demo
Total count= 11268431
wenjun@2023150001:~/intergrated_exp1$ ./no-mutex-demo
Total count= 11302571
wenjun@2023150001:~/intergrated_exp1$ ./no-mutex-demo
Total count= 9641147
wenjun@2023150001:~/intergrated_exp1$
```

图 58 no-mutex-demo 的输出

在修复后的 mutex-demo 代码中, 通过 pthread_mutex_init()初始化互斥锁, 并在执行 count++这一极度脆弱的临界区代码首尾, 精准包裹了 pthread_mutex_lock()与 pthread_mutex_unlock()操作。

核心逻辑代码片段如下:

```
pthread_mutex_t m; //增加一个互斥量
pthread_mutex_init(&m, NULL); //初始化互斥量

// 临界区访问保护机制
pthread_mutex_lock(&m);
count++;
pthread_mutex_unlock(&m);
```

图 59 mutex-demo.c

但在初步测试加锁版本的 mutex-demo 时, 发现输出结果依然呈现不确定的随机波动 (如 33435711、

33533191 等), 经过对指导书提供的代码流进行深度溯源, 捕捉到了一个经典的“**主线程越权数据竞争**” Bug。

在原始代码的线程创建阶段:

```
for (t = 0; t < thread_num; t++) {  
  
    count = 0; // 缺陷代码: 主线程在循环内对共享变量发生越权清零  
  
    pthread_create(...);  
  
}
```

隐患剖析: 当早期创建的子线程已经成功获取互斥锁并在临界区内安全递增 count 时, 尚未休眠的主线程在继续执行循环, 并直接在未申请互斥锁的裸奔状态下执行了 count = 0; 赋值操作, **主线程的特权覆盖直接击穿了子线程的锁保护屏障, 导致早期线程的累加进度被随机吞噬**, 完美反证了并发安全铁律: 同一共享变量的所有读写访问方 (含主线程) 必须处于统一的锁体系约束之下。

将共享变量初始化语句 count = 0; 剥离至线程生成循环之外, 确保在多执行流产生前完成资源初始化。修复此逻辑缺陷后再次多次运行程序, 16 个线程基于底层互斥量机制实现了完美的排他性临界区访问。程序最终输出被绝对稳定地锁定在理论正确值 33554432, 无可辩驳地深度验证了操作系统底层临界区保护与严格时序管理的重要工程价值。

连续三次运行结果:

```
wenjun@2023150001:~/intergrated_exp1$ gcc -o mutex-demo mutex-demo.c -lpthread  
wenjun@2023150001:~/intergrated_exp1$ ./mutex-demo  
Total count= 33554432  
wenjun@2023150001:~/intergrated_exp1$ ./mutex-demo  
Total count= 33554432  
wenjun@2023150001:~/intergrated_exp1$ ./mutex-demo  
Total count= 33554432  
wenjun@2023150001:~/intergrated_exp1$
```

图 60 多次运行结果一致可靠的运行成功画面

五、实验体会: (根据自己情况填写)

通过两个操作系统预备实验, 我深刻体会到了 Linux 底层进程与线程管理的精妙设计, 成功将课堂上抽象的操作系统理论转化为了直观的系统级行为。在第一部分的进程控制实验中, 借助 fork 与 exec 函数族的实际编码, 我完整观测了进程从分裂、变身到最终销毁的生命周期, 并结合代码时序成功复现了僵尸进程与孤儿进程的底层接管机理。特别是通过读取 /proc/slabinfo 内核统计信息, 我非常直观地看到了多进程与多线程在 task_struct 与 mm_struct 等数据结构分配上的巨大数量差异, 这种真切的数据对比让我彻底厘清了操作系统将“调度执行”与“资源分配”进行解耦的核心工程意义。

在第二部分的进程间通信与同步实验中, 我不仅熟练掌握了管道、共享内存以及 System V IPC 机制的特性与数据流转过程, 更在多线程并发实操中获得了极其宝贵的工程排错经验。在验证互斥锁保护临界区的环节, 我遇到了“已加锁但结果依然随机错乱”的隐蔽现象, 经过深度溯源与逻辑推演, 最终揪出了主线程在循环中越权清零共享变量的致命逻辑缺陷。这一排错过程让我深刻认识到, 并发编程的安全绝不仅仅是简单地调用加锁 API, 而是必须在宏观架构上确保所有的并发执行流严格处于统一的锁体系约束之下, 这种对内核调度不确定性与严格时序管理的敬畏之心, 是我在此次综合实验中取得的最宝贵财富。

最后在实现 Shell 实验中在终端调试时, 我曾误用后台执行符连接编译与运行指令, 导致前台总拉起旧版程序。查阅作业管理机制后, 我改用逻辑短路控制符确保程序按序同步拉起, 深刻体会到系统交互背后的状态机时序哲学。

处理管道命令切割时，起初定长截断导致参数携带脏空格从而引发报错。通过勾勒内存拓扑图，我直接对查找函数返回的指针地址覆写结束符实施物理截断，并精准偏移指针安全提取纯净参数，真正领悟了底层指针直接操控内存的高效性。

在实现起止标签打印时我遭遇了输出流污染陷阱。最初将打印逻辑置于子进程变身前，引入管道后发现左侧提示神秘消失。重审源码才醒悟该子进程的标准输出已被重定向至通道写端，打印内容被错误输送给了对端进程。将打印任务剥离回独立的父进程后才彻底解决界面错乱。

最令我后怕的是失效进程的收割。原以为命令拼错导致变身失败便会自然终止，极限测试时却引发双进程抢夺键盘的灾难。结合进程分裂理论我惊醒：替换失败的子进程仍保留着原有的死循环程序，这让我把终结失效子进程的防漏铁律彻底刻进了编程直觉。

指导教师批阅意见：

成绩评定：

指导教师签字：

年 月 日

备注：